

UAV Control Challenge

Sistema de Control y Monitoreo de Dron DJI Tello con ROS 2 en Docker

Miguel Mateo Bermeo Montero

Facultad de Ingeniería

Departamento de Telecomunicaciones

Universidad de Cuenca

mateo.bermeo@ucuenca.edu.ec

Freddy Eduardo López Padilla

Facultad de Ingeniería

Departamento de Telecomunicaciones

Universidad de Cuenca

freddy.lopez@ucuenca.edu.ec

Tyrone Miguel Novillo Bravo

Facultad de Ingeniería

Departamento de Telecomunicaciones

Universidad de Cuenca

tyrone.novillo@ucuenca.edu.ec

Resumen—El presente informe documenta el desarrollo de un sistema modular de control y monitoreo remoto para un vehículo aéreo no tripulado (UAV) DJI Tello, implementado sobre una arquitectura ROS2 Jazzy dentro de un contenedor Docker. El sistema se compone de cuatro nodos funcionales: un conector de comunicación bidireccional con el dron (`drone_connector`), un monitor de telemetría en tiempo real (`telemetry_monitor`), un planificador de misiones autónomas con interfaz gráfica PyQt5 e integrado con lógica de seguridad *failsafe* (`mission_planner`), y un detector de objetos basado en visión computacional mediante OpenCV (`object_detector`). La comunicación entre nodos se realiza mediante tópicos ROS2 con mensajes estándar, mientras que la interacción con el hardware del dron se centraliza en un único nodo que emplea la librería `djitellopy` y un modelo de concurrencia basado en `MultiThreadedExecutor` con grupos de `callbacks` diferenciados. La contenerización con Docker y la orquestación mediante Docker Compose garantizan un entorno de ejecución reproducible y portable.

Index Terms—ROS 2, Docker, DJI Tello, UAV, OpenCV, telemetría.

I. INTRODUCCIÓN

Las redes de sensores inalámbricos (WSN) han evolucionado hasta integrar plataformas móviles capaces de captar información del entorno de forma autónoma. En este contexto, un dron equipado con cámara y sensores inerciales puede actuar como un nodo sensor móvil dentro de una arquitectura distribuida, aportando datos visuales y de telemetría que complementan la información de nodos fijos.

El presente proyecto tiene como finalidad el desarrollo de un sistema de control y monitoreo para el dron DJI Tello, utilizando ROS 2 como *middleware* de comunicación y Docker como plataforma de contenerización. ROS 2 proporciona una arquitectura basada en nodos y tópicos que permite desacoplar las funcionalidades del sistema: comunicación con el hardware, visualización de telemetría, planificación de misiones, detección de objetos y seguridad de vuelo. Cada funcionalidad se encapsula en un nodo independiente que se comunica a través de tópicos, lo cual facilita el desarrollo, las pruebas y el mantenimiento.

Docker permite empaquetar toda la infraestructura de software, incluyendo ROS 2, librerías de visión computacional y dependencias del SDK del dron, en una imagen única que garantiza la reproducibilidad del entorno. Esta decisión de diseño es especialmente relevante en el contexto académico, donde múltiples equipos requieren entornos idénticos para el desarrollo y la evaluación de sus proyectos.

II. MARCO TEÓRICO

II-A. ROS 2 (*Robot Operating System 2*)

ROS 2 es un conjunto de bibliotecas y herramientas de software para el desarrollo de aplicaciones robóticas. A diferencia de su predecesor, ROS 2 se construye sobre el estándar DDS (*Data Distribution Service*), lo que le confiere comunicación descentralizada, soporte nativo para calidad de servicio (QoS) y compatibilidad con sistemas en tiempo real. Su arquitectura se fundamenta en los siguientes conceptos:

Nodos: unidades de procesamiento independientes que realizan una tarea específica. Cada nodo puede publicar y suscribirse a múltiples tópicos.

Tópicos: canales de comunicación unidireccionales basados en el patrón publicador/suscriptor. Los mensajes se tipan mediante definiciones estándar como `std_msgs/String`, `sensor_msgs/Image` o `std_msgs/Int32`.

Executors y Callback Groups: ROS 2 permite la ejecución concurrente de `callbacks` mediante `MultiThreadedExecutor`. Los `MutuallyExclusiveCallbackGroup` garantizan que los `callbacks` del grupo nunca se ejecuten simultáneamente, mientras que los `ReentrantCallbackGroup` permiten la ejecución en paralelo de sus miembros.

II-B. Docker

Docker es una plataforma de contenerización que permite empaquetar aplicaciones junto con todas sus dependencias en unidades aisladas denominadas contenedores. Un `Dockerfile` define la receta de construcción de la imagen, y `docker-compose` permite orquestar la creación y configuración de contenedores de forma declarativa. En el contexto

de ROS 2, el modo de red `host` (`network_mode: host`) es esencial para que el protocolo de descubrimiento DDS funcione correctamente, ya que éste depende de multicast UDP en la red local.

II-C. OpenCV y espacio de color HSV

OpenCV es una biblioteca de código abierto para visión computacional en tiempo real. Para la detección de objetos por color, se emplea la conversión del espacio BGR a HSV (*Hue, Saturation, Value*), que resulta más robusto frente a variaciones de iluminación. La segmentación se realiza mediante `cv2.inRange()` para generar máscaras binarias, seguida de operaciones morfológicas (*opening* y *closing*) para reducir el ruido, y la detección de contornos con `cv2.findContours()`.

II-D. DJI Tello

El DJI Tello es un micro-dron de 87 g diseñado para uso educativo e interior. Cuenta con un sistema de posicionamiento visual, una cámara de 5 megapíxeles capaz de transmitir video en vivo a 720p (1280×720 a 30 fps), y una batería LiPo de 1100 mAh con autonomía aproximada de 13 minutos. Opera en la banda de 2.4 GHz y su velocidad máxima es de 28.8 km/h. El dron expone un SDK basado en comandos UDP: el cliente envía comandos de texto al puerto 8889 del dron (192.168.10.1) y recibe respuestas de estado y stream de video por puertos dedicados [1]. La librería Python `djitellopy` encapsula este protocolo, proporcionando una API de alto nivel para control de vuelo, lectura de telemetría y captura de video.

II-E. cv_bridge

`cv_bridge` es un paquete de ROS 2 que permite convertir entre mensajes de tipo `sensor_msgs/Image` y arreglos de OpenCV (`numpy.ndarray`). Se utiliza `imgmsg_to_cv2()` para la conversión de mensaje a imagen y `cv2_to_imgmsg()` para la conversión inversa.

III. DESARROLLO

III-A. Arquitectura del sistema

El sistema se compone de cuatro nodos ROS 2 que se comunican a través de siete tópicos, todos ejecutándose dentro de un único contenedor Docker con red en modo `host`. La Fig. 6 muestra el diagrama de la arquitectura completa.

Se tomó la decisión de diseño de eliminar el nodo `video_viewer` propuesto originalmente en la guía del proyecto, dado que el nodo `object_detector` ya implementa la visualización del video en tiempo real dentro de su ventana de OpenCV, haciendo redundante un nodo dedicado exclusivamente a esa tarea. Asimismo, la lógica de `battery_failsafe` (Nodo 4 original) se integró directamente dentro del `mission_planner`, ya que la validación de batería antes del despegue y el aterrizaje de emergencia están íntimamente acoplados con la lógica de ejecución de misiones.

La Tabla I resume los tópicos del sistema, indicando el tipo de mensaje, el nodo publicador y los nodos suscriptores.

Cuadro I
TÓPICOS ROS 2 DEL SISTEMA

Tópico	Tipo	Pub	Sub
/battery_status	Int32	connector	monitor, planner
/telemetry	String (JSON)	connector	monitor
/drone_status	String	connector	planner
/tello_image	Image	connector	detector
/tello_cmd	String (JSON)	planner	connector
/tello_cmd_priority	String (JSON)	planner	connector
/objects_count	Int32	detector	—

III-B. Nodo 1: drone_connector

Este nodo constituye el único punto de contacto entre el sistema ROS 2 y el hardware del dron. Ningún otro nodo interactúa directamente con el Tello; toda la comunicación pasa por `drone_connector`. Sus responsabilidades son: establecer y mantener la conexión UDP con el dron, publicar periódicamente datos de telemetría, batería, estado y video, y recibir y ejecutar comandos de vuelo provenientes del `mission_planner`.

III-B1. Modelo de concurrencia: hilos y grupos de callbacks: El diseño concurrente de este nodo es crítico para el funcionamiento seguro del sistema. La función `main()` instancia un `MultiThreadedExecutor` con 6 hilos, lo que permite que múltiples callbacks se ejecuten en paralelo. Sin embargo, no todos los callbacks deben ejecutarse simultáneamente; la librería `djitellopy` no es *thread-safe*, y dos lecturas concurrentes al dron pueden corromper la comunicación UDP. Por esta razón, se definen tres grupos de callbacks con políticas de concurrencia diferenciadas:

1) timers_group (MutuallyExclusiveCallbackGroup): agrupa los cuatro timers de publicación periódica: `publish_battery` (cada 1 s), `publish_telemetry` (cada 0.5 s), `publish_status` (cada 1 s) y `publish_image` (cada 1/30 s $\approx$ 30 fps). La política *mutuamente exclusiva* garantiza que estos cuatro timers nunca se ejecuten en paralelo entre sí. Esto es necesario porque todos acceden al objeto `self.drone` para leer datos del Tello. Si, por ejemplo, `publish_image` estuviera leyendo un frame mientras `publish_telemetry` consulta la altura, ambas operaciones competirían por el socket UDP interno de `djitellopy`, causando respuestas corruptas o *timeouts*.

2) cmd_group (ReentrantCallbackGroup): contiene el suscriptor de `/tello_cmd` (comandos normales de misión). La política *reentrante* permite que este callback se ejecute en paralelo con los timers. Esto es esencial porque las funciones de movimiento de `djitellopy` (`move_forward()`, `rotate_clockwise()`, etc.) son **bloqueantes**: cada llamada tarda entre 3 y 5 segundos en retornar. Si este callback estuviera en el mismo grupo mutuamente exclusivo que los timers, un comando de movimiento bloquearía la publicación de telemetría y video durante toda su duración.

3) priority_group (ReentrantCallbackGroup): contiene el suscriptor de `/tello_cmd_priority` (coman-

dos urgentes del *failsafe*). Al estar en un grupo reentrante separado, este callback puede ejecutarse **inmediatamente** incluso cuando hay un comando normal bloqueante en curso. Cuando la batería cae por debajo del umbral crítico, el *mission_planner* publica un comando de aterrizaje en este tópico. El callback activa una bandera de aborto (`self.aborted`, protegida con `threading.Lock`) y envía el comando `land` directamente al dron usando `send_command_without_return()`, que transmite el paquete UDP sin esperar respuesta, garantizando un aterrizaje inmediato.

III-B2. Publicación periódica de datos: Los cuatro timers alimentan al resto del sistema con datos actualizados del dron:

`/battery_status` publica el porcentaje de batería como `Int32` cada segundo. `/telemetry` publica un JSON con altura, velocidades en tres ejes (x, y, z), orientación (pitch, roll, yaw), barómetro, tiempo de vuelo y temperaturas cada 0.5s. `/drone_status` publica una cadena con el estado actual del dron: `DISCONNECTED`, `LANDED` o `FLYING`. `/tello_image` publica los frames de la cámara como `sensor_msgs/Image` a 30 fps, convirtiendo de RGB a BGR con OpenCV y luego a mensaje ROS2 con `cv_bridge`.

III-B3. Doble canal de comandos: Los comandos se reciben en formato JSON (`{cmd": "forward", "value": 50}`). El canal normal (`/tello_cmd`) interpreta el comando y llama a la función correspondiente de *djitellopy*. El canal prioritario (`/tello_cmd_priority`) activa la bandera de aborto, que bloquea cualquier comando normal pendiente, y envía el aterrizaje directamente por UDP sin pasar por la API bloqueante.

III-C. Nodo 2: *telemetry_monitor*

Este nodo es solo de lectura y visualización en consola. Se suscribe a dos tópicos: `/battery_status` (donde cachea el último valor de batería) y `/telemetry` (donde parsea el JSON y muestra los datos). La visualización emplea secuencias de escape ANSI (`\033[2J\033[H]`) para limpiar la terminal y redibujar el dashboard en cada actualización, creando un efecto de actualización en tiempo real sin acumulación de líneas. La Fig. 1 muestra la interfaz del monitor de telemetría.

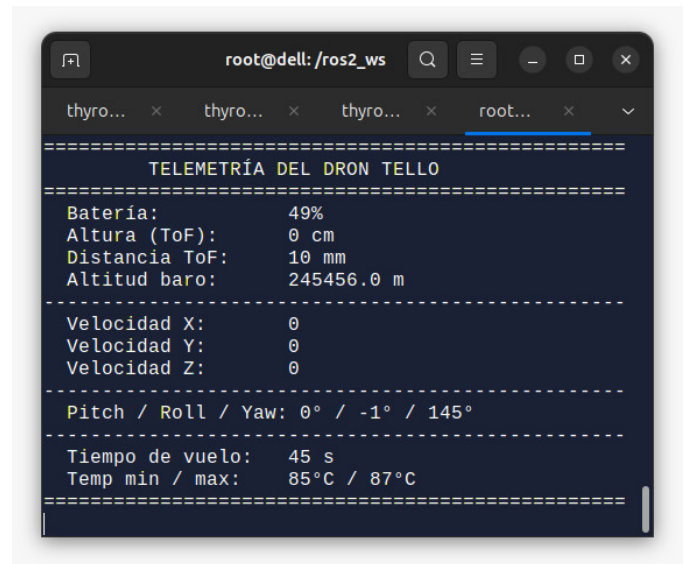


Figura 1. Dashboard de telemetría en consola mostrando batería, altura, velocidades, orientación y temperatura.

III-D. Nodo 3: *mission_planner* (con *failsafe* integrado)

Este nodo combina tres funcionalidades: una interfaz gráfica (GUI) construida con PyQt5 para diseñar misiones, un motor de ejecución secuencial de pasos, y la lógica de seguridad *failsafe* por batería crítica. La comunicación entre el hilo de ROS2 y el hilo de la GUI de Qt se realiza mediante señales (`pyqtSignal`), ya que Qt no permite la manipulación de widgets desde hilos secundarios.

III-D1. Interfaz gráfica: La GUI permite al usuario construir una secuencia de pasos de misión arrastrando y reordenando elementos en una lista. Cada misión comienza obligatoriamente con un despegue y termina con un aterrizaje (pasos fijos no eliminables). Entre ellos, el usuario puede añadir movimientos lineales, rotaciones, flips, comandos `go` (movimiento 3D con velocidad), `curve` (trayectoria curva), `stop` (hover) y `emergency`. Las Figs. 2 y 3 muestran la interfaz principal y el diálogo de añadir paso.

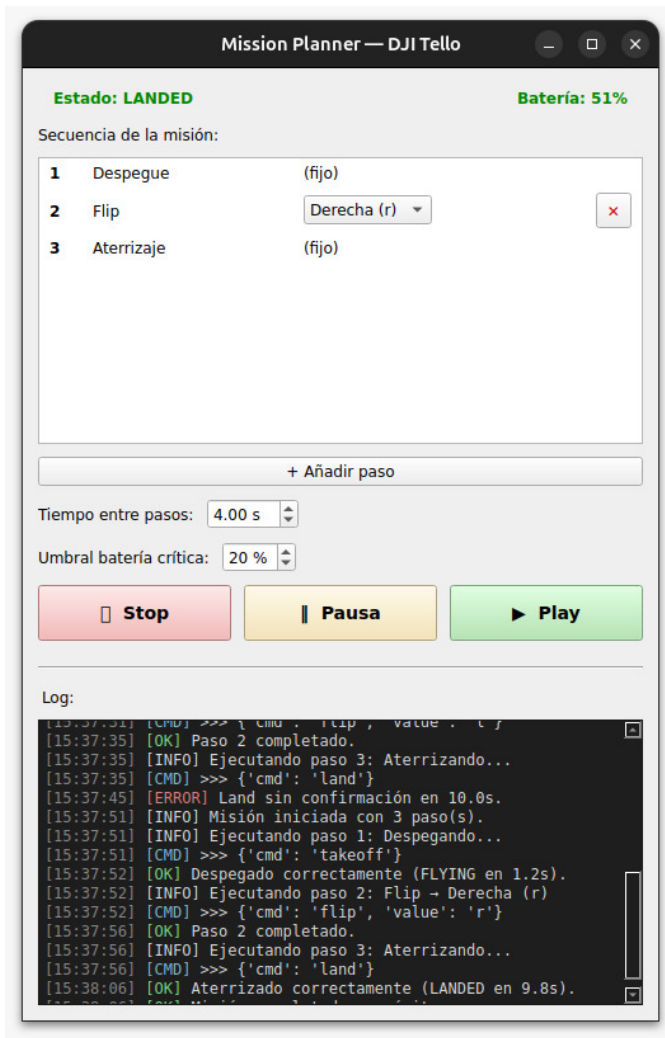


Figura 2. Interfaz principal del Mission Planner mostrando la secuencia de pasos, controles de ejecución y log en tiempo real.

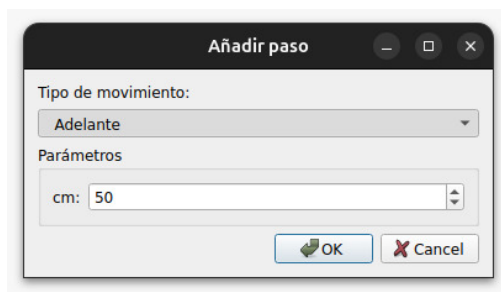


Figura 3. Diálogo para añadir un paso a la misión con selección de comando y parámetros.

III-D2. Motor de ejecución: Un timer ROS2 a 10Hz (tick) implementa una máquina de estados que avanza paso a paso por la secuencia de la misión. Para takeoff y land, el motor espera confirmación del estado del dron (FLYING o LANDED respectivamente), con un *timeout* de 10 segundos tras el cual aborta. Para comandos de movimiento, el motor

espera un tiempo configurable entre pasos (por defecto 4 s) antes de avanzar al siguiente.

III-D3. Failsafe de batería: El callback `_check_battery_failsafe()` se ejecuta cada vez que se recibe una actualización de batería. Si el dron está volando (FLYING) y la batería cae por debajo del umbral configurable desde la GUI, se publica un comando de aterrizaje en `/tello_cmd_priority`, se aborta la misión en curso y se registra el evento en el log. Además, antes de iniciar cualquier misión, se valida que la batería esté por encima del umbral, cancelando el despegue si no se cumple la condición.

III-E. Nodo 4: *object_detector*

Este nodo implementa la detección y conteo de objetos de color rojo y negro en el flujo de video del dron. Se suscribe a `/tello_image` y publica el conteo total en `/objects_count`. El procesamiento de cada frame sigue la secuencia: suavizado gaussiano, conversión a HSV, generación de máscaras por rango de color, operaciones morfológicas (opening y closing con kernel 7×7), detección de contornos, y filtrado por área mínima (800 px) y compacidad (50 % del bounding box).

Para el color rojo se emplean dos rangos HSV (H: 0–10 y H: 165–180) ya que el rojo envuelve el extremo del canal de matiz. Para el negro se exige V bajo (≤ 40) y S bajo (≤ 60) simultáneamente, lo que descarta sombras y superficies oscuras con color saturado. Todos los umbrales son parámetros ROS2 ajustables en tiempo de ejecución. Las Figs. 4 y 5 muestran el detector en funcionamiento.

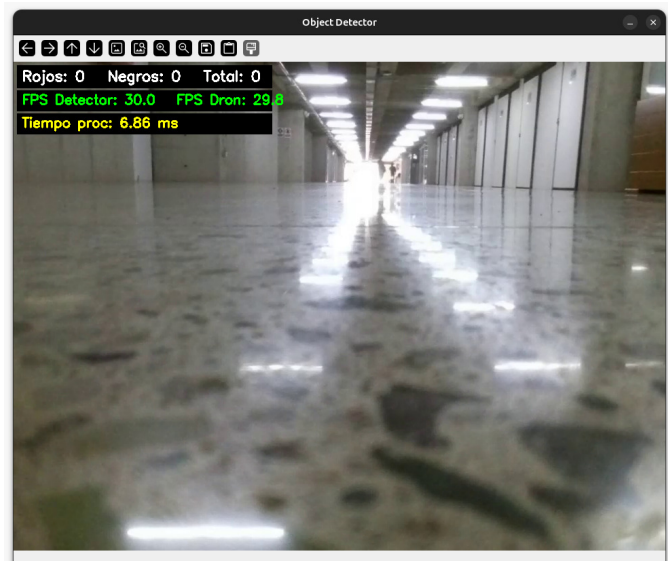


Figura 4. Object Detector sin objetos de interés: se muestran las métricas de FPS y tiempo de procesamiento.

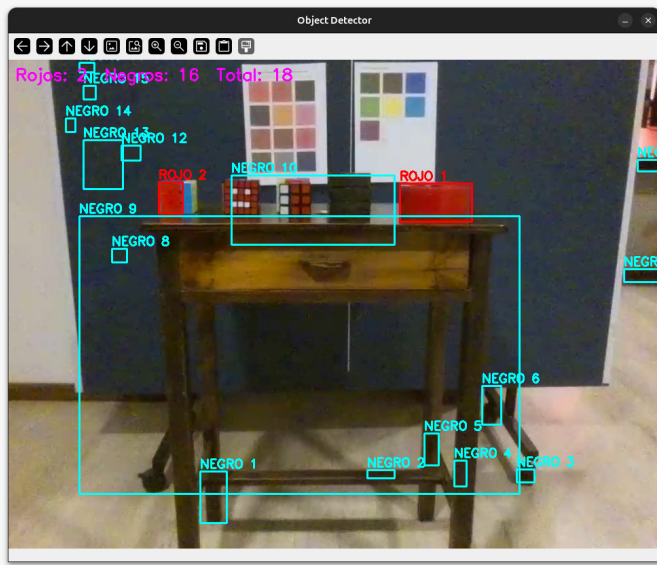


Figura 5. Object Detector con detección activa: 2 objetos rojos y 16 negros identificados con bounding boxes.

III-F. Contenerización con Docker

III-F1. Dockerfile: La imagen se construye sobre `osrf/ros:jazzy-desktop` e instala las dependencias del sistema (`python3-opencv`, `python3-pyqt5`, `ros-jazzy-cv-bridge`), la librería `djitellopy` mediante `pip`, y copia el workspace de ROS2 para compilarlo con `colcon build`. Un `entrypoint.sh` automatiza el `sourcing` de los archivos `setup.bash` de ROS2 y del workspace.

Listing 1. Dockerfile del proyecto

```
1 FROM osrf/ros:jazzy-desktop
2 ENV DEBIAN_FRONTEND=noninteractive
3 ENV PIP_BREAK_SYSTEM_PACKAGES=1
4
5 RUN apt-get update && apt-get install -y \
6     python3-pip python3-opencv \
7     python3-pyqt5 ros-jazzy-cv-bridge \
8     x11-apps && rm -rf /var/lib/apt/lists/*
9
10 RUN pip install djitellopy \
11     --ignore-installed numpy && \
12     pip install "numpy<2" --force-reinstall
13
14 WORKDIR /ros2_ws
15 COPY src/ src/
16 RUN /bin/bash -c \
17     "source /opt/ros/jazzy/setup.bash && \
18     colcon build --packages-select tello_control"
19
20 COPY entrypoint.sh /entrypoint.sh
21 RUN chmod +x /entrypoint.sh
22 ENTRYPOINT ["/entrypoint.sh"]
23 CMD ["bash"]
```

Es importante destacar la línea `pip install "numpy<2--force-reinstall`: la imagen base incluye `numpy 1.26.4` instalado por `apt`, y `djitellopy` arrastra `numpy 2.x` como dependencia. Sin embargo, `cv_bridge` fue compilado contra `numpy 1.x` y es incompatible con la versión 2. Esta línea fuerza un downgrade a una versión compatible.

III-F2. Docker Compose: El archivo `docker-compose.yml` orquesta la creación del contenedor con todas las variables de entorno necesarias para el reenvío de la interfaz gráfica X11 y el lanzamiento automático de todos los nodos:

Listing 2. `docker-compose.yml`

```
1 services:
2   tello_drone:
3     image: tello_ros2:latest
4     container_name: tello_drone
5     network_mode: host
6     environment:
7       - DISPLAY=${DISPLAY}
8       - QT_X11_NO_MITSHM=1
9       - XDG_RUNTIME_DIR=/tmp
10      - QT_QPA_PLATFORM=xcb
11      - LIBGL_ALWAYS_SOFTWARE=1
12     volumes:
13       - /tmp/.X11-unix:/tmp/.X11-unix
14     devices:
15       - /dev/dri:/dev/dri
16     stdin_open: true
17     tty: true
18     command: ros2 launch tello_control tello_launch.py
```

La variable `LIBGL_ALWAYS_SOFTWARE=1` fuerza el renderizado por CPU, necesario porque el contenedor no tiene acceso directo a la GPU del host. El `network_mode: host` comparte la pila de red del host con el contenedor, lo que permite la comunicación UDP con el Tello (conectado vía WiFi) y el descubrimiento DDS entre nodos.

III-F3. Despliegue: Para desplegar el sistema se requiere: conectar la máquina host a la red WiFi del Tello (TELLO-XXXXXX), habilitar el acceso X11 con `xhost +local:docker`, y ejecutar `docker compose up`. La imagen está disponible en Docker Hub para su descarga directa.

IV. CONCLUSIONES

- El sistema desarrollado demuestra la viabilidad de emplear ROS2 como middleware de comunicación para el control de UAVs en un contexto de redes de sensores. La arquitectura basada en nodos desacoplados permitió distribuir las responsabilidades de manera clara: un único nodo centraliza la comunicación con el hardware, mientras que los demás consumen y procesan los datos publicados.
- El diseño del modelo de concurrencia en `drone_connector`, con tres grupos de callbacks diferenciados, resultó esencial para garantizar la seguridad del vuelo. La separación entre comandos normales y prioritarios, combinada con el uso de `send_command_without_return()` para el aterrizaje de emergencia, permite reaccionar ante situaciones críticas sin esperar a que finalicen operaciones bloqueantes en curso.
- La integración de la lógica de failsafe dentro del `mission_planner` simplificó la arquitectura al reducir un nodo sin perder funcionalidad. De manera similar, la eliminación del `video_viewer` en favor del `object_detector` evitó la redundancia de dos ventanas de visualización de video.

- La contenerización con Docker demostró ser una solución efectiva para garantizar la reproducibilidad del entorno, aunque presentó desafíos específicos como los conflictos de versiones de numpy entre paquetes de apt y pip, y la configuración del reenvío X11 para las interfaces gráficas.

REFERENCIAS

- [1] Ryze Tech, "Tello User Manual v1.4," 2018. Disponible en: <https://www.ryzerobotics.com/tello>
- [2] Open Robotics, "ROS 2 Documentation: Jazzy Jalisco," 2024. Disponible en: <https://docs.ros.org/en/jazzy/>
- [3] D. Zucker, "DJITelloPy: DJI Tello drone Python interface using the official Tello SDK," GitHub, 2023. Disponible en: <https://github.com/damiafuentes/DJITelloPy>
- [4] OpenCV Team, "OpenCV: Open Source Computer Vision Library," 2024. Disponible en: <https://opencv.org/>
- [5] Docker Inc., "Docker Documentation," 2024. Disponible en: <https://docs.docker.com/>

Diagrama de arquitectura

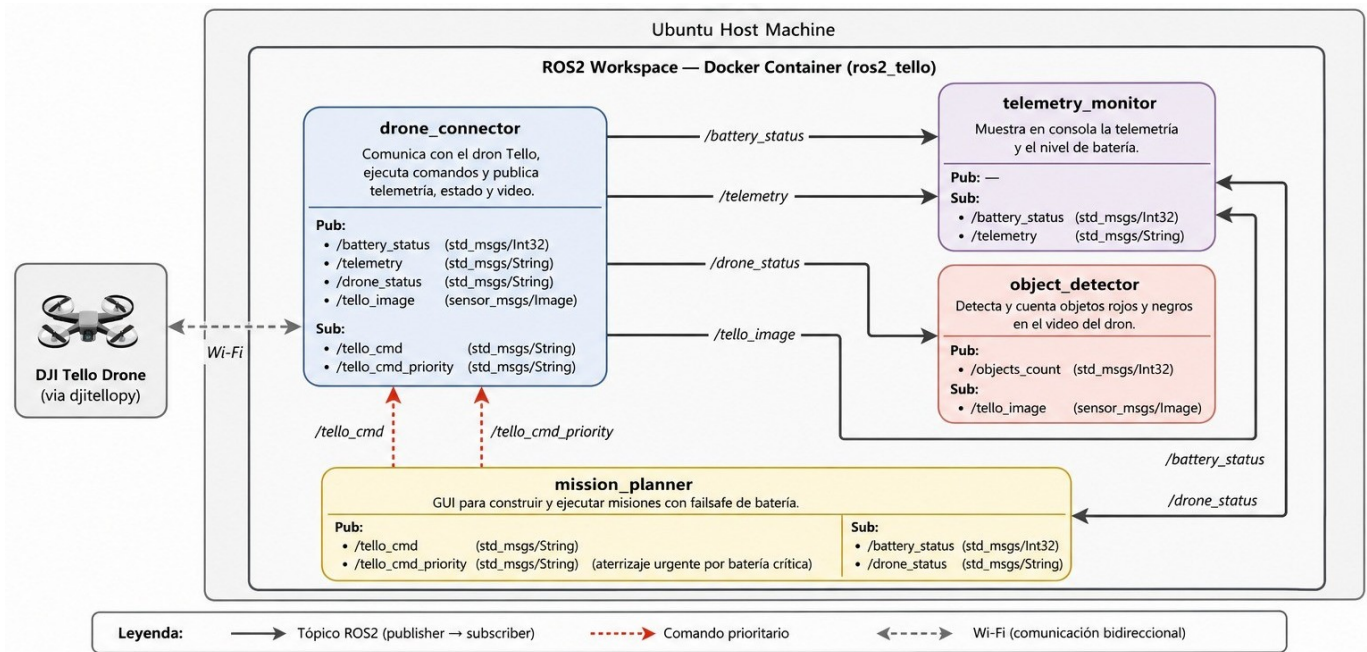


Figura 6. Arquitectura del sistema: nodos ROS2, tópicos y comunicación con el dron Tello.

Repositorio GitHub

Los códigos fuente ROS, así como el docker-compose se encuentran disponibles en el repositorio GitHub del proyecto: <https://github.com/thyron001/uav-control>



Figura 7. Código QR del repositorio GitHub.

Docker Hub

A continuación se indica el comando para obtener la imagen creada:

```
docker pull miguelbermeo1/tello_ros2
```